百瑞互联
BARROT

Barrot Confidential

BR1601 SDK 说明

2025. 2.21

V1.4

变更记录

版本	变更内容	日期	修改人
1.0	初始版本	2023-03-16	田磊鑫
1.1	变更命名	2023-06-29	ICSAE
1.2	加上2.4G应用描述	2024-01-02	ICSAE
1.3	修改2.4G应用接口函数	2024-02-04	ICSAE
1.4	增加关于MAC地址说明	2025-02-21	田磊鑫

Barrot Confidential

目录

1. 概述 8
2. 引脚定义 8
3. SDK 目录结构 13
3.1. 硬件架构 14
3.2. 软件架构 15
4. 开发工具 15
4.1. 固件下载 15
4.2. 工具准备 16
4.2.1. 手机端 16
4.2.2. PC 端 16
4.3. 设置串口 16
4.4. 扫描连接 17
5. BLE DTSS/DTSC 说明 20
5.1. DTSS 初始化概述 20
5.2. DTSS 蓝牙应用层初始化 21
5.2.1. GAP 事件回调函数添加 21
5.2.2. 广播参数初始化 22
5.2.3. 蓝牙服务初始化 23
5.3. DTSS 广播控制 24
5.3.1. 广播开启 24

Barrot Confidential

5.3.2. 广播关闭 24
5.4. DTSS 消息处理 24
5.4.1. 蓝牙消息 24
5.5. DTSS 发送数据流 25
5.5.1. 串口接收数据 25
5.5.2. 通过蓝牙发送数据 27
5.6. DTSS 接收数据流 28
5.6.1. 蓝牙数据接收 28
5.6.2. 发送数据到串口 29
5.7. DTSS 蓝牙服务 31
5.7.1. 服务初始化 31
5.7.2. 服务的响应 33
5.8. DTSC 初始化概述 33
5.9. DTSC 蓝牙应用层初始化 33
5.10. DTSC 蓝牙事件处理 34
5.10.1. 报告扫描数据 34
5.10.2. 创建蓝牙连接 34
5.10.3. 蓝牙断开事件 36
5.11. DTSC 收发数据 36
5.12. 任务与调度机 37
6. API 说明 37
6.1. ble 功能模块 37

Barrot Confidential

6.1.1. le_adv_add 37
6.1.2. le_adv_remove 39
6.1.3. le_adv_clear 39
6.1.4. le_adv_enable 40
6.1.5. le_adv_disable 40
6.1.6. le_scan_enable 41
6.1.7. le_scan_disable 42
6.1.8. le_conn_create 42
6.1.9. le_conn_destory 44
6.1.10. gap_conn_create_by_addr 44
6.1.11. gap_ev_subscribe 45
6.1.12. gap_ev_unsubscribe 46
6.1.13. gatts_svc_create 46
6.1.14. gatts_chr_add 47
6.1.15. gatts_dsc_ccc_add 48
6.1.16. gatts_chr_val_write_rsp 48
6.1.17. gatts_chr_val_read_rsp 49
6.1.18. gatts_chr_val_notif 50
6.1.19. gattc_svc_discover 50
6.1.20. gattc_chr_discover 51
6.1.21. gattc_des_discover 51
6.1.22. gattc_dsc_ccc_disable 52

Barrot Confidential

6.1.23. gattc_dsc_ccc_enable 52
6.1.24. gattc_chr_val_read 53
6.1.25. gattc_chr_val_write 54
6.1.26. gattc_chr_val_write_cmd 54
6.2. 2.4G 功能模块 55
6.2.1. direct_24g_init 55
6.2.2. direct_24g_tx 56
6.2.3. direct_24g_rx 56
6.2.4. cbk_direct_24g_tx 57
6.2.5. cbk_direct_24g_rx 57
6.2.6. cbk_direct_24g_rx_timeout 57
6.2.7. 错误代码 58
6.3. 调试信息模块 58
6.3.1. co_log_init 58
6.3.2. co_printf 59
6.3.3. co_printf_type 59
6.3.4. dbg_type 60
6.4. 休眠管理模块 60
6.4.1. br_sleep_lock_acquire 60
6.4.2. br_sleep_lock_release 61
6.4.3. br_sleep 61
6.4.4. app_sleep_prepare_proc 62

Parrot Confidential

6.4.5. app_sleep_resume_proc....62
6.5. 阻塞延迟模块....62
6.5.1. delay_cycles....63
6.5.2. delay_n_us....63
6.5.3. delay_n_10us....63
6.5.4. delay_n_100us....64
6.5.5. delay_n_ms....64
6.6. 杂类接口....64
6.6.1. 查询 SDK 版本....64
7. 公司简介....65
8. 联系我们....65
8.1. 北京....65
8.2. 深圳....65
8.3. 上海....66
9. 版权....66

Barrot Confidential

1. 概述

BR1601 系列芯片是支持蓝牙 BLE 5.1 版本的系统级芯片,包含一颗 32 位高性能低功耗处理器，主频高达 64MHz，内置 48K 字节休眠可保持 SRAM，256K 字节 Flash 程序存储器，具有低功耗，高性能特性。

BR1601 系列芯片集成先进的 BLE5.1 射频收发器，符合蓝牙 BLE5.1 规范，可配置为标准的 1Mbps BLE 模式，2Mbps 增强 BLE 模式，125kbps BLE 长距离模式(S8)，500kbps BLE 长距离模式(S2)。支持 AOA（到达角）和 AOD（离去角），支持 RSSI（接收信号强度指示)，支持主从角色，支持多连接，支持数据包长度扩展，支持 KEYSCAN，IRC，10 位 1.33Msps ADC(可配置为 16 位 16Ksps)，支持模拟 MIC 输入，PGA 放大，支持基本、通用、高级定时器 TIMER，RTC，WWDG，IWDG，LPUART，USART，SPI，I2C 等外设。

2. 引脚定义

表 1 类型符号表

符号	描述
I	输入
O	输出
I/O	双向
OD	开漏
P	电源
G	地

Barrot Confidential

AIO	模拟端口

表 2 引脚定义表

编号	管脚名称 (默认功能)	类型	复用功能	功能描述
1	PA2	I/O	SPI1_MOSI(I2S1_SD) KEY3	-
2	PA3	I/O	SPI1_MISO(I2S1_MCK) KEY4	-
3	RESET	AIO	-	-
4	PB0	I/O	TIM1_CH1 SPI2_NSS(I2S2_WS) USART1_RTS LPUART_RTS KEY11	-
5	PA4 (SWDCLK)	I/O	TIM1_CH3N USART1_TXD(7816_TX1) KEY9	-
6	PA5 (SWDIO)	I/O	TIM1_ETR USART1_RXD(7816_RST1) KEY10	-
7	PA6	I/O	TIM1_BKIN	-

Barrot Confidential

			USART2_TXD USART1_CK(7816_CLK1) KEY5	
8	VDD_FLASH	P	-	外挂2.2uF电容
9	PB8 (X32K_I)	I/O	TIM1_CH1 IIC_SDA USART1_RTS KEY7	ADC3
10	PB9 (X32K_O)	I/O	TIM1_CH2 IIC_SCL USART1_CTS KEY8	ADC2
11	PB1	I/O	TIM1_CH2 SPI2_CLK(I2S2_CLK) USART1_CTS LPUART_TXD KEY12 ANT_SW4	-
12	PB2	I/O	TIM1_CH3 SPI2_MOSI LPUART_RXD KEY13	-

Barrot Confidential

			ANT_SW5	
13	PB3	I/O	TIM1_CH4 SPI2_MISO(I2S_MCK) LPUART_CTS PA_LDO_EN ANT_SW1	WAKEUP
14	PB4	I/O	SPI2_CLK(I2S2_CLK) TIM3_CH1(IRC_TX) USART2_TXD(7816_TX2) ANT_SW6	-
15	PB5	I/O	SPI2_MISO(I2S2_MCK) TIM3_CH2(IRC_RX) USART2_RXD(7816_RST2) RCC_MCO ANT_SW7	-
16	PB6	I/O	SPI2_MOSI(I2S2_SD) TIM3_CH3 IIC_SDA USART1_TXD SWDCLK ANT_SW2	ADC5
17	PB7	I/O	SPI2_NSS(I2S2_WS)	ADC4

Barrot Confidential

			TIM3_CH4 IIC_SCL USART1_RXD SWDIO ANT_SW3	
18	PB10	I/O	TIM1_CH3 LPUART_RTS USART2_RXD IIC_SMBA KEY6	ADC1
19	PB12	I/O	TIM1_CH1N LPUART_TXD USART2_CTS	AMIC_BIAS
20	PB11	I/O	TIM1_CH4 LPUART_RXD USART2_RTS IIC_SMBA	AMIC_N
21	PB13	I/O	TIM1_CH2N LPUART_CTS USART2_CK	AMIC_P
22	VCC	P	-	芯片电源
23	SWITCH	P	-	DCDC外部接口

Barrot Confidential

24	VDCDC	P	-	DCDC输出
25	RFIOP	AIO	-	天线端口
26	VDD_PA	P	-	PA电源
27	VDCDCRF	P	-	DCDC输出，与 VDCDC接在一起
28	VCCRF	P	-	芯片电源
29	X32M_O	AIO	GPIO31	外部32MHz晶体
30	X32M_I	AIO	GPIO32	外部32MHz晶体
21	PA0	I/O	SPI1_NSS(I2S1_WS) KEY1	-
32	PA1	I/O	SPI1_CLK(I2S1_CLK) KEY 2	-
-	Ground PAD	P	-	地

3.SDK 目录结构

SDK 大体目录结构：

```
| ├──bsp
|    ├──common
|    ├ ─core
|    └────driver
| ├──cs
|    ├──inc
```

Barrot Confidential

```
|   └──src
├──demo
|   ├──ant
|   ├──ble
|   ├──mesh
|   └──peripheral
├──doc
├──tools
|   └──mdk
```

图 1 SDK 目录

bsp：启动文件与外设驱动。

cs：蓝牙协议栈库。

demo：例程目录。

doc：说明文档。

tools/mdk：下载烧录算法

系统架构[kw1]

3.1. 硬件架构

图 2 硬件架构

Barrot Confidential

3.2.软件架构

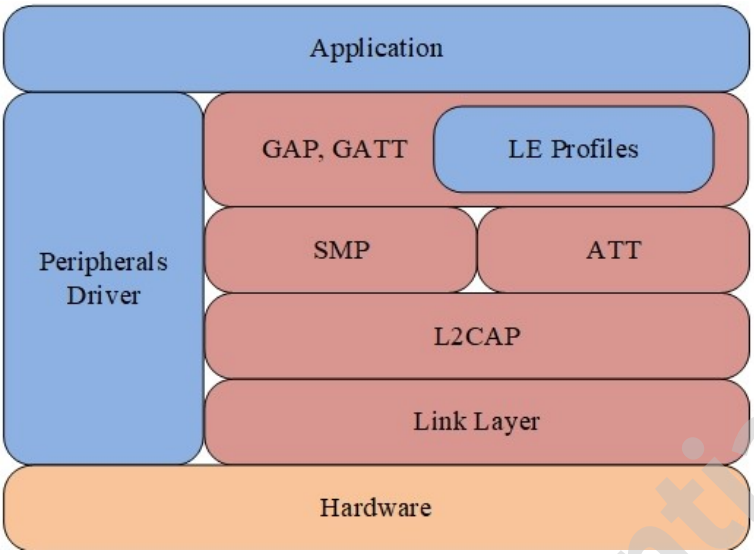


图 3 软件架构

4.开发工具

4.1. 固件下载

编译后通过 JLINK 或者类似工具下载。接线方式如下：

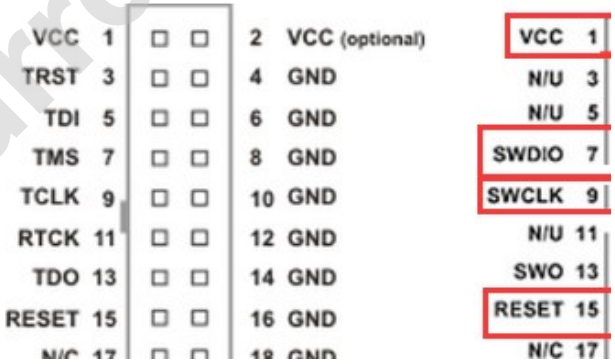


图 4 接线方式

JLINK-SWD 对应信号分别连接芯片 VCC，SWDIO(PA5)，SWCLK(PA4)，GND，并接上 RESET 信号。

Barrot Confidential

4.2.工具准备

4.2.1. 手机端

IOS：lightblue,nrf_connect

Android:nrf_connect

安装上述软件并确保软件对蓝牙有访问权限。

4.2.2.PC 端

串口通讯工具，如 sscom 等

USB 转串口板

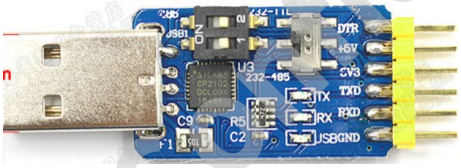


图 5 USB 转串口板

4.3.设置串口

将芯片的 PB6(UART_TX),PB7(UART_RX),GND 连接串口工具板(RX,TX,GND)，波特率 115200。打开对应串口，复位芯片，SSCOM 收到打印提示。

Barrot Confidential

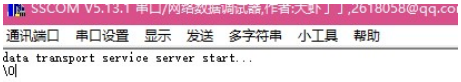




图 6 串口设置

4.4.扫描连接

以 android nrf_connect 软件为例：

- 在 SCANNER 界面搜索 BR_DTSS 设备

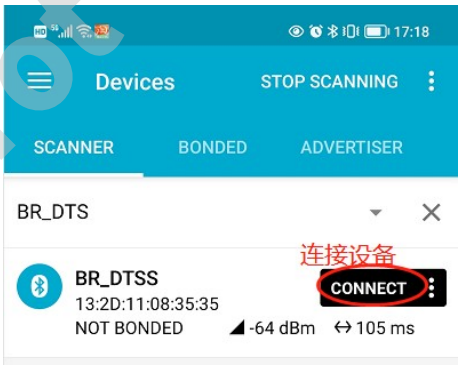


图 7 搜索界面

- 连接设备

Barrot Confidential

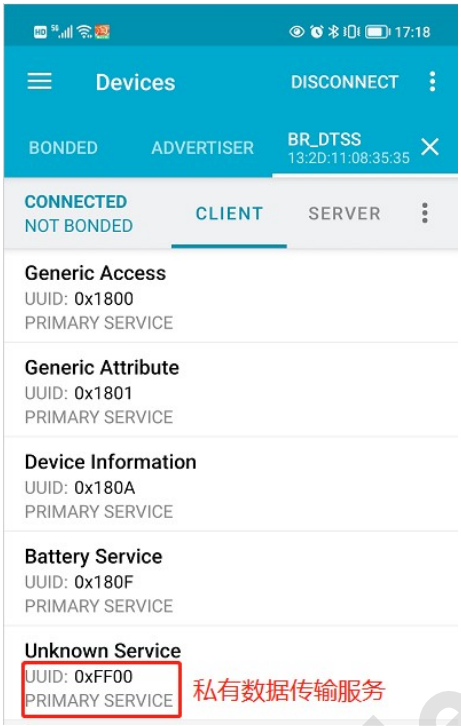


图 8 连接设备

- 打开接收通道



图 9 打开接收通道

- APP 向串口发送数据

Barrot Confidential



图 10 进入发送界面

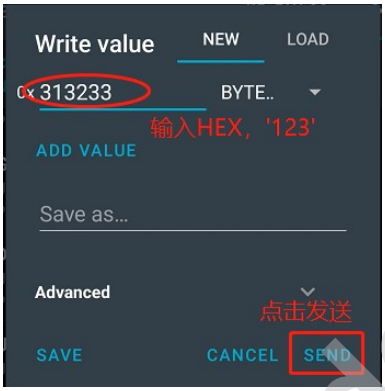


图 11 发送数据

- 串口向发 APP 送数据

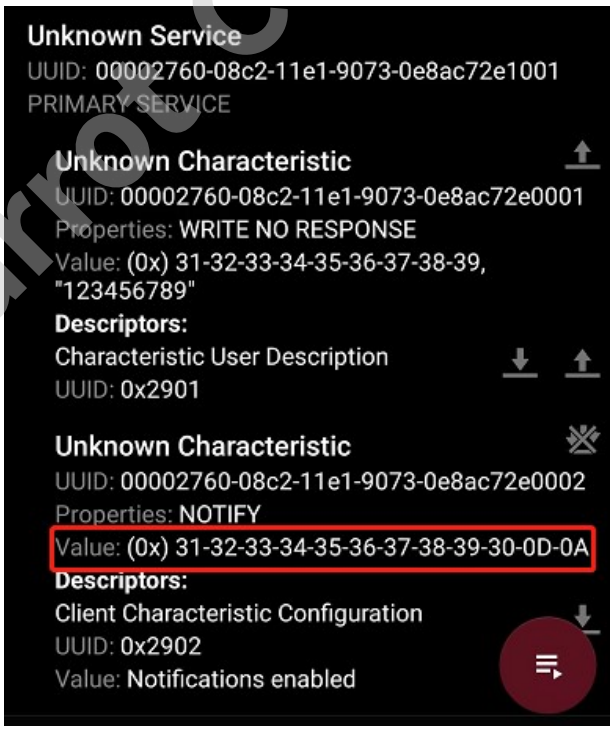


图 12 串口向发 APP 送数据

Barrot Confidential

- 上述操作涉及到了 Service，Characteristic，UUID 概念，也有 Write No Response, Notify 概念。对于理解 BLE 蓝牙的核心概念与工作原理有重要的意义。BLE 的应用就是蓝牙服务（Service）及组合。蓝牙服务分为标准服务和自定义服务。服务的本质是对服务下特征（Characteristic）的读写操作。

5.BLE DTSS/DTSC 说明

DTSS/DTSC 例程是基于 BLE 的数据透明传输例程，在此加以说明，便于开发者理解代码。

5.1. DTSS 初始化概述

初始化过程包括：日志功能初始化，蓝牙协议栈初始化，蓝牙应用层初始化，数据串口初始化。

其中，蓝牙协议栈初始化（ble_stack_init()执行时），首先读取 MAC 地址，如果没有 MAC 地址，则根据 FLASH UUID 虚拟一个 04 7F 0E XX XX XX 的 MAC 地址。

如果对此 MAC 地址有要求，量产时 MAC 有几种写入途径：

- 由用户开发固件调用 API，在产测环节写入；
- 通过百瑞的 BRMP 工具在产测环节写入；
- 联系百瑞销售，芯片或模组出厂前写入。

Barrot Confidential

```
int main(void)
{
    delay_n_10us(50*1000);

    co_log_init();

    co_printf(DEMO_STRING);

    ble_stack_init();
    ble_gap_init();
    ble_adv_init();

    ble_dtss_init();

    ble_adv_start();

    app_usart_dma_enable(ENABLE);
    usart_tx_dma_send((uint8_t*)DEMO_STRING, sizeof(DEMO_STRING));
    delay_n_10us(500);
    app_usart_dma_enable(DISABLE);

    while (1)
    {
        cs_schedule();
    }
}
```

图 13 DTSS 初始化

5.2.DTSS 蓝牙应用层初始化

5.2.1. GAP 事件回调函数添加

```
static gap_ev_cbk_t ble_gap_ev_cbk[] = {
    {GAP_EV_CONN_CMPL            , (gap_cbk_t *)ble_gap_connected         } ,
    {GAP_EV_DISC_CMPL            , (gap_cbk_t *)ble_gap_disconnected      } ,
    {GAP_EV_LE_CONN_UPDATE_CMPL  , (gap_cbk_t *)ble_gap_conn_update       } ,
    {GAP_EV_LE_PEER_CONN_PAR_REQ , (gap_cbk_t *)ble_gap_peer_conn_par_req } ,
};

int ble_gap_init(void)
{
    gap_ev_subscribe(ble_gap_ev_cbk, sizeof(ble_gap_ev_cbk)/sizeof(gap_ev_cbk_t));

    return 0;
}
```

图 14 GAP 事件订阅

Barrot Confidential

5.2.2.广播参数初始化

```
int ble_adv_init(void)
{
    uint8_t adv_dat[] = {
        0x02, 0x01, 0x04,
        0x05, 0x03, 0x00, 0x18, 0x01, 0x18,
        0x0A, 0x09, 'B', 'R', '1', '6', '0', '1', 'A', 'x', 'x',
        0x03, 0x19, 0x00, 0x00,
    };

    adv_data_t adv_dat_seg = {
        .data = adv_dat,
        .len  = sizeof(adv_dat),
    };

    adv_param_t adv_par = {
        .adv_hdl             = 0x01,
        .adv_ev_prop         = (ADV_EV_PROP_BITS_LEGACY
                                | ADV_EV_PROP_BITS_SCANNABLE
                                | ADV_EV_PROP_BITS_CONNECTABLE
                               ),
        .prim_adv_intv_min   = 0x20,
        .prim_adv_intv_max   = 0x40,
        .prim_adv_ch_map     = (ADV_CH_MAP_37 | ADV_CH_MAP_38 | ADV_CH_MAP_39),
        .own_addr_type       = ADDR_TYPE_PUBLIC,
        .peer_addr_type      = ADDR_TYPE_PUBLIC,
        .peer_addr           = {0},
        .adv_filter_policy   = ADV_POLICY_PROC_SCAN_CONN_ALL,
        .adv_tx_pwr          = 0x7F,
        .prim_adv_phy        = ADV_PHY_1M,
        .scnd_adv_max_skip   = 0x00,
        .scnd_adv_phy        = ADV_PHY_1M,
        .adv_sid             = 0,
        .scan_req_notif_en   = false,
        .adv_data_frag_pref  = ADV_FRAG_PREF_CNTL_MAY_FRAG,
        .adv_data_frag_num   = 1,
        .adv_data_frag       = &adv_dat_seg,
        .scn_data_frag_pref  = ADV_FRAG_PREF_CNTL_MAY_FRAG,
        .scn_data_frag_num   = 0,
        .scn_data_frag       = NULL,
    };

    le_adv_add(&adv_par, NULL, NULL);

    return 0;
}
```

图 15 广播参数初始化

此函数中设置了广播数据：

```
int ble_adv_init(void)
{
    uint8_t adv_dat[] = {
        0x02, 0x01, 0x04,
        0x05, 0x03, 0x00, 0x18, 0x01, 0x18,
        0x0A, 0x09, 'B', 'R', '1', '6', '0', '1', 'A', 'x', 'x',
        0x03, 0x19, 0x00, 0x00,
    };
```

图 16 广播/扫描响应数据

Barrot Confidential

0x0B:长度，共 11 字节，是后面数据的总长

长度计算：0x0B　= 11 = 1 + 10

注意：广播/扫描响应数据必须按此格式（长度，类型，内容）填写，否则对端无法识别。

5.2.3.蓝牙服务初始化

```
int ble_dtss_init(void)
{
    uuid_t svc_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF00)
    };
    dtss_svc = gatts_svc_create(&svc_uuid, dtss_svc_cbk, NULL);

    uuid_t chr0_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF01)
    };
    dtss_chr0 = gatts_chr_add(dtss_svc, &chr0_uuid
                            , GATT_CHR_PROP_WRITE_WITHOUT_RSP
                            , (ATT_PERM_READ | ATT_PERM_WRITE)
                            , dtss_chr0_cbk, NULL);

    uuid_t chr1_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF02)
    };
    dtss_chr1 = gatts_chr_add(dtss_svc, &chr1_uuid
                            , GATT_CHR_PROP_NOTIFY
                            , ATT_PERM_READ
                            , dtss_chr1_cbk, NULL);

    gatts_dsc_ccc_add(dtss_chr1
                    , GATT_CHR_PROP_NOTIFY
                    , ATT_PERM_READ | ATT_PERM_WRITE);


    gatts_svc_start(dtss_svc);

    return 0;
}
```

图 17 蓝牙服务初始化

Barrot Confidential

5.3.DTSS 广播控制

5.3.1. 广播开启

广播控制模块初始化后，调用 ble_adv_start()启动广播。

5.3.2.广播关闭

广播控制模块初始化后，调用 ble_adv_stop()停止广播。

5.4.DTSS 消息处理

5.4.1. 蓝牙消息

蓝牙消息处理函数添加位置如下，蓝牙消息主要包括蓝牙连接，断开，参数更新等，需用户并添加回调函数到 ble_gap_ev_cbk 表中。

```
static gap_ev_cbk_t ble_gap_ev_cbk[] = {
    {GAP_EV_CONN_CMPL           , (gap_cbk_t *)ble_gap_connected         } ,
    {GAP_EV_DISC_CMPL           , (gap_cbk_t *)ble_gap_disconnected      } ,
    {GAP_EV_LE_CONN_UPDATE_CMPL , (gap_cbk_t *)ble_gap_conn_update       } ,
    {GAP_EV_LE_PEER_CONN_PAR_REQ , (gap_cbk_t *)ble_gap_peer_conn_par_req } ,
};
```

图 18 蓝牙消息处理函数

其他消息如下，应用层根据需要添加对应的处理函数：

Barrot Confidential

```
typedef enum gap_event {
    GAP_EV_ADV_SET_ADD_CMPL = 0x01, /** adv_add_cmpl_t
                                        Advertising Set added complete */
    GAP_EV_ADV_SET_RMV_CMPL,        /** adv_rmv_cmpl_t
                                        Advertising Set reomved complete */
    GAP_EV_ADV_SET_CLR_CMPL,        /** adv_clr_cmpl_t
                                        Clear all Advertising Sets complete */
    GAP_EV_ADV_SET_EN_CMPL,         /** adv_en_cmpl_t
                                        Advertising Set Enable complete */
    GAP_EV_ADV_SET_DIS_CMPL,        /** adv_dis_cmpl_t
                                        Advertising Set Disable complete */

    GAP_EV_SCN_STARTED,             /** scn_start_t
                                        Scan started */
    GAP_EV_SCN_STOPED,              /** scn_stop_t
                                        Scan stoped */
    GAP_EV_ADV_REPORT,              /** adv_rpt_t
                                        Scanned Advertising report */

    GAP_EV_CONN_CMPL,               /** gap_conn_cmpl_t */
    GAP_EV_DISC_CMPL,               /** gap_disc_cmpl_t */

    GAP_EV_LE_CONN_UPDATE_CMPL,     /** le_peer_conn_update_cmpl_t */
    GAP_EV_LE_PEER_CONN_PAR_REQ,    /** le_peer_conn_par_req_t */

} gap_event_t;
```

图 19 其他蓝牙消息类型

回调函数中传入的参数类型跟消息类型相关，如连接完成:

```
/**
 * The parameter of GAP_EV_DISC_CMPL
 */
typedef struct gap_disc_cmpl {
    peer_t   peer;   /** Peer device instance. */
    uint8_t  result;
} gap_disc_cmpl_t;
```

图 20 连接事件参数

5.5.DTSS 发送数据流

发送数据流指向 BR1601 串口发送数据，传输到 APP 的数据流。

5.5.1. 串口接收数据

主体流程如下:

- 串口接收数据触发 DMA 中断，串口接收超时触发 UART 中断。
- DMA 初始化开启两个中断，达到 1/2 接收数量和指定数量传输完成中断。

Barrot Confidential

- DMA 中断，UART 接收超时中断，触发将数据读取到环形缓冲区中。

```
/**
 * @brief  dma irq handler
 */
void DMA_Channel1_2_3_4_IRQHandler(void)
{
    //TX
    if(DMA_GetFlagStatus(DMA_FLAG_TC1, DMA))
    {
        //TX Transfer complete interrupt
        usart_sending = false;
        DMA_ClearFlag(DMA_FLAG_TC1, DMA);
    }
    //RX
    if(DMA_GetFlagStatus(DMA_FLAG_TC2, DMA))
    {
        //RX Transfer complete interrupt
        DMA_ClearFlag(DMA_FLAG_TC2, DMA);
        usart_rx_check_in_irq();

    }
    if(DMA_GetFlagStatus(DMA_FLAG_HT2, DMA))
    {
        //RX Half transfer interrupt
        DMA_ClearFlag(DMA_FLAG_HT2, DMA);
        usart_rx_check_in_irq();
    }
}
```

图 21 DMA_Channel1_2_3_4_IRQHandler()

说明：

DMA 接收数据存储在 USART_rx_dma_buf[USART_RX_DMA_SIZE]

usart_rx_check_in_irq()将 DMA 存储位置数据搬运到下面位置：

usart_rx_fifo_buf[USART_RX_FIFO_SIZE]。

Barrot Confidential

```
void usart_rx_check_in_irq(void)
{
    uint16_t rx_pos;

    rx_pos = USART_RX_DMA_SIZE - DMA_GetCurrDataCounter(USARTx_Rx_DMA_Channel);

    if(rx_pos <  rx_old_pos)
    {
        app_usart_rx_data_fifo_enter(&USART_rx_dma_buf[rx_old_pos],(USART_RX_DMA_SIZE - rx_old_pos));

        if(rx_pos > 0)
        {
            app_usart_rx_data_fifo_enter(&USART_rx_dma_buf[0],rx_pos);
        }

        rx_old_pos = rx_pos;
    }
    else if(rx_pos >  rx_old_pos)
    {
        app_usart_rx_data_fifo_enter(&USART_rx_dma_buf[rx_old_pos],(rx_pos - rx_old_pos));
        rx_old_pos = rx_pos;
    }
    else if(rx_pos != rx_old_pos)
    {
        //error
        rx_old_pos = rx_pos;
    }

}
```

图 22 usart_rx_check_in_irq()

5.5.2.通过蓝牙发送数据

app_usart_rx_data_fifo_enter 将调用 usart_forward_to_ble_loop()发送数据：

```
uint8_t app_usart_rx_data_fifo_enter(const uint8_t *p_data, uint16_t len)
{
    uint32_t in_len;
    //store data in fifo
    while(len)
    {
        if(usart_rx_fifo_in >= usart_rx_fifo_out )
        {
            in_len = USART_RX_FIFO_SIZE-usart_rx_fifo_in;
            if(in_len > len)
            {
                in_len = len;
            }
            memcpy(&usart_rx_fifo_buf[usart_rx_fifo_in],p_data,in_len);
            len = len-in_len;
            p_data += in_len;
            usart_rx_fifo_in = (usart_rx_fifo_in + in_len)%USART_RX_FIFO_SIZE;

        }
        else if(usart_rx_fifo_in < usart_rx_fifo_out )
        {
            in_len = usart_rx_fifo_out-usart_rx_fifo_in-1;
            if(in_len > len)
            {
                in_len = len;
            }
            memcpy(&usart_rx_fifo_buf[usart_rx_fifo_in],p_data,in_len);
            len = len-in_len;
            usart_rx_fifo_in = (usart_rx_fifo_in + in_len)%USART_RX_FIFO_SIZE;

            //fifo full,drop the rest data
            if(len)
            {
            }
            break;
        }
    }

    if(!ble_sending)
    {
        usart_forward_to_ble_loop();
    }
```

Barrot Confidential

图 23 app_usart_rx_data_fifo_enter()

以上函数调用 gatts_chr_val_notif()将数据发送到手机 APP。

```
void usart_forward_to_ble_loop(void)
{
    uint32_t in_temp;
    uint16_t ble_send_len;

    in_temp = usart_rx_fifo_in;
    if(usart_rx_fifo_out < in_temp)
    {
        ble_send_len = in_temp-usart_rx_fifo_out;
    }
    else if(usart_rx_fifo_out > in_temp){
        ble_send_len = USART_RX_FIFO_SIZE-usart_rx_fifo_out;
    }
    else if(usart_rx_fifo_out == in_temp){
        // fifo empty, stop send loop
        ble_sending = false;
        return;
    }
    if(ble_send_len > ble_att_mtu)
    {
        ble_send_len = ble_att_mtu;
    }
    if(ble_send_len == 2)
    {
        if(memcmp( &usart_rx_fifo_buf[usart_rx_fifo_out],"\xfa\x55",2) == 0)
        {
            usart_rx_fifo_out = usart_rx_fifo_in = 0;
            return;
        }
    }

    gatts_chr_val_notif(dtss_peer,dtss_chr,&usart_rx_fifo_buf[usart_rx_fifo_out], ble_send_len);
    usart_rx_fifo_out = (usart_rx_fifo_out+ble_send_len)%USART_RX_FIFO_SIZE;

    return;
}
```

图 24 数据发送到手机 APP

5.6.DTSS 接收数据流

接收数据流是指蓝牙主设备发送到 BR1601 的数据，通过中间环节最终到达串口。

5.6.1. 蓝牙数据接收

对蓝牙的特征值写入数据，将引发 dtss_chr0_cbk 回调：

```
void dtss_chr0_cbk(gatt_event_t event, void *context, void *param)
{
    switch (event) {
        case GATTS_EV_CHR_VALUE_WRITE_REQ: {
            gatt_chr_val_write_req_t *in = (gatt_chr_val_write_req_t *)param;
            co_printf("Chr0 value write %D %X\n", in->val_len, *in->val);
            gatts_chr_val_write_rsp(in->peer, in->chr, GATT_OK);

            app_usart_tx_fifo_enter(in->val,in->val_len);

            break;
        }
    }
}
```

Bariot Confidential

图 25 dtss_chr0_cbk

此函数将调用 app_usart_rx_fifo_enter()数据送入缓冲区：

usart_tx_fifo_buf[USART_TX_FIFO_SIZE] ；此函数填充完数据缓冲区后，执行app_usart_tx_process()函数。

```
uint8_t app_usart_tx_fifo_enter(const uint8_t *p_data, uint16_t len)
{
    uint32_t in_len, out_temp;

    out_temp = usart_tx_fifo_out;
    while(len)
    {
        if(usart_tx_fifo_in >= out_temp )
        {
            in_len = USART_TX_FIFO_SIZE-usart_tx_fifo_in;
            if(in_len > len)
            {
                in_len = len;
            }
            memcpy(&usart_tx_fifo_buf[usart_tx_fifo_in],p_data,in_len);
            len = len-in_len;
            p_data +=  in_len;
            usart_tx_fifo_in = (usart_tx_fifo_in + in_len)%USART_TX_FIFO_SIZE;

        }
        else if(usart_tx_fifo_in < out_temp )
        {
            in_len = out_temp-usart_tx_fifo_in-1;
            if(in_len > len)
            {
                in_len = len;
            }
            memcpy(&usart_tx_fifo_buf[usart_tx_fifo_in],p_data,in_len);
            len = len-in_len;
            usart_tx_fifo_in = (usart_tx_fifo_in + in_len)%USART_TX_FIFO_SIZE;

            //fifo full,drop the rest data
            if(len)
            {
            }
            break;
        }
    }

    app_usart_tx_process();
    if(usart_tx_fifo_out == 0)app_usart_tx_process();
    return len;
}
```

图 26 app_usart_tx_fifo_enter()

5.6.2. 发送数据到串口

缓冲区的数据通过 usart_tx_dma_send()触发发送：

Barrot Confidential

```
void app_usart_tx_process(void)
{
    uint32_t in_temp,len;
    uint8_t *p_data;

    in_temp = usart_tx_fifo_in;
    if(usart_tx_fifo_out < in_temp)
    {
        len = in_temp-usart_tx_fifo_out;
        p_data = &usart_tx_fifo_buf[usart_tx_fifo_out];
        if(usart_tx_dma_send(p_data,len) == true)
        {
            usart_tx_fifo_out = in_temp;
        }

    }
    else if(usart_tx_fifo_out > in_temp)
    {
        len = USART_TX_FIFO_SIZE-usart_tx_fifo_out;
        p_data = &usart_tx_fifo_buf[usart_tx_fifo_out];
        if(usart_tx_dma_send(p_data,len) == true)
        {
            usart_tx_fifo_out = 0;
        }

    }
}
```

图 27 app_usart_tx_process()

usart_tx_dma_send()将判断是否正在发送，如果当前没有正在传输的 dma 事务，则启动一次传输。

Barrot Confidential

```
uint8_t usart_tx_dma_send(uint8_t *p_data, uint16_t len)
{
    DMA_InitType DMA_InitStructure;

    if(usart_sending)
    {
        return false;
    }
    /* USARTx_Tx_DMA_Channel (triggered by USARTx Tx event) Config */
    DMA_InitStructure.PeriphAddr     = USARTx_DAT_Base;
    DMA_InitStructure.MemAddr        = (uint32_t)p_data;
    DMA_InitStructure.Direction      = DMA_DIR_PERIPH_DST;
    DMA_InitStructure.BufSize        = len;
    DMA_InitStructure.PeriphInc      = DMA_PERIPH_INC_DISABLE;
    DMA_InitStructure.DMA_MemoryInc  = DMA_MEM_INC_ENABLE;
    DMA_InitStructure.PeriphDataSize = DMA_PERIPH_DATA_SIZE_BYTE;
    DMA_InitStructure.MemDataSize    = DMA_MemoryDataSize_Byte;
    DMA_InitStructure.CircularMode   = DMA_MODE_NORMAL;
    DMA_InitStructure.Priority       = DMA_PRIORITY_VERY_HIGH;
    DMA_InitStructure.Mem2Mem        = DMA_M2M_DISABLE;
    DMA_Init(USARTx_Tx_DMA_Channel, &DMA_InitStructure);

    usart_sending = true;
    DMA_EnableChannel(USARTx_Tx_DMA_Channel, ENABLE);

    return true;
}
```

图 28 usart_tx_dma_send()

5.7.DTSS 蓝牙服务

5.7.1. 服务初始化

在应用层，用户可以根据图 41 中的例子添加所需的服务。

Barrot Confidential

```
int ble_dtss_init(void)
{
    uuid_t svc_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF00)
    };
    dtss_svc = gatts_svc_create(&svc_uuid, dtss_svc_cbk, NULL);

    uuid_t chr0_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF01)
    };
    dtss_chr0 = gatts_chr_add(dtss_svc, &chr0_uuid
                            , GATT_CHR_PROP_WRITE_WITHOUT_RSP
                            , (ATT_PERM_READ | ATT_PERM_WRITE)
                            , dtss_chr0_cbk, NULL);

    uuid_t chr1_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF02)
    };
    dtss_chr1 = gatts_chr_add(dtss_svc, &chr1_uuid
                            , GATT_CHR_PROP_NOTIFY
                            , ATT_PERM_READ
                            , dtss_chr1_cbk, NULL);

    gatts_dsc_ccc_add(dtss_chr1
                    , GATT_CHR_PROP_NOTIFY
                    , ATT_PERM_READ | ATT_PERM_WRITE);


    gatts_svc_start(dtss_svc);

    return 0;
}
```

图 29 ble_dtss_init()

注：添加新的服务请按上述方法编写 ble_xxx_init()，并在 main 函数中调用。

服务的 UUID 及特征的 UUID 根据用户需求自行定义。

服务的创建使用 gatts_svc_create()函数，参数是包含服务的 UUID，及回调函数，还有一个保留参数。

特征的添加使用 gatts_chr_add()函数，服务及特征的回调函数在添加服务和特征时添加。

Barrot Confidential

5.7.2.服务的响应

通过手机 APP 对尾数对 UUID FF01 特征写入数据，会触发 dts_chr0_cbk 回调，应用层在此接收数据。

5.8. DTSC 初始化概述

初始化过程包括：日志功能初始化，蓝牙协议栈初始化，蓝牙应用层初始化，数据串口初始化。

```
int main(void)
{
    delay_n_10us(50*1000);

    co_log_init();

    co_printf(DEMO_STRING);

    ble_stack_init();
    ble_gap_init();

    ble_scan_init();

    app_usart_dma_enable(ENABLE);
    usart_tx_dma_send((uint8_t*)DEMO_STRING, sizeof(DEMO_STRING));
    delay_n_10us(500);
    app_usart_dma_enable(DISABLE);

    while (1)
    {
        cs_schedule();
    }
}
```

图 30 DTSC 初始化

5.9.DTSC 蓝牙应用层初始化

注册蓝牙连接与断开回调函数：

```
static gap_ev_cbk_t ble_gap_ev_cbk[] = {
    {GAP_EV_CONN_CMPL           , (gap_cbk_t *)ble_gap_connected        } ,
    {GAP_EV_DISC_CMPL           , (gap_cbk_t *)ble_gap_disconnected     } ,
};

int ble_gap_init(void)
{
    gap_ev_subscribe(ble_gap_ev_cbk, sizeof(ble_gap_ev_cbk)/sizeof(gap_ev_cbk_t));

    return 0;
}
```

Barrot Confidential

初始化蓝牙扫描参数并启动扫描：

```
int ble_scan_init(void)
{

    scn_phy_param_t phy_par = {
        .scn_type = SCN_TYPE_ACTIVE,
        .scn_intv = 160,
        .scn_win= 150
    };
    scn_param_t san_par = {
        .filter_duplicates = false,
        .duration = 0x0000,
        .period   = 0x0000,
//          .own_addr_type = 0x00,
        .filter_policy = SCN_POLICY_BASIC_UNFILTERED,
        .phys = SCH_PHY_BITS_1M,
        .phys_par = &phy_par,
    };

    le_scan_enable(&san_par, (gap_cbk_t *)ble_scan_rsp_data_cbk, NULL);

    return 0;
}
```

5.10. DTSC 蓝牙事件处理

5.10.1. 报告扫描数据

解析并连接目标设备：

```
void ble_scan_rsp_data_cbk(gap_event_t ev, void *ctx, adv_rpt_t *arg)
{
    uint8_t addr[] = {0x11,0x22,0x33,0x33,0x22,0x33};
//    co_printf("%s %d\r\n",__FUNCTION__,__LINE__);

    if(memcmp(addr,&(arg->addr[0]),6) == 0){
        for(uint8_t i=0;i<6;i++){
            co_printf("%02x ",arg->addr[i]);
        }
        co_printf("\r\n");
        le_scan_disable();
        gap_conn_create_by_addr(arg->addr,arg->addr_type,ble_scan_dev_to_conn_cbk,NULL);
    }

}
```

上述函数搜到目标设备后，创建蓝牙连接。

5.10.2. 创建蓝牙连接

建立连接后，触发服务查找：

Boriot Confidential

```
void ble_gap_connected(gap_event_t ev, void *ctx, gap_conn_cmpl_t *arg)
{
    app_usart_dma_enable(ENABLE);
    co_printf("conn complete, %08X, %02X\r\n", (uint32_t)arg->peer, arg->result);
    uuid_t svc_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF00)
    };
    gattc_svc_discover(arg->peer,&svc_uuid,(gatt_cbk_t *)ble_disc_ff00_svc_cbk,NULL);
}
```

服务查的成功后触发特征值查找：

```
void ble_disc_ff00_svc_cbk(gap_event_t ev, void *ctx, gatt_svc_discovered_t *arg)
{
    co_printf("%s %d uuid:%04x\r\n",__FUNCTION__,__LINE__,arg->uuid.val.uuid16);
    uuid_t chr0_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF01)
    };
    uuid_t chr1_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0xFF02)
    };

    if(arg->uuid.val.uuid16 == 0xFF00){
        gattc_chr_discover(arg->peer,arg->svc,&chr0_uuid,(gatt_cbk_t *)ble_disc_ff01_chr_cbk,NULL);
        gattc_chr_discover(arg->peer,arg->svc,&chr1_uuid,(gatt_cbk_t *)ble_disc_ff02_chr_cbk,NULL);
    }
}
```

特征值查找完成后，触发特征值描述符查找：

```
void ble_disc_ff02_chr_cbk(gap_event_t ev, void *ctx, gatt_chr_discovered_t *arg)
{
    uuid_t des0_uuid = {
          UUID_LEN_16
        , UUID16_VAL_SET(0x2902)
    };
    co_printf("%s %d uuid:%04x\r\n",__FUNCTION__,__LINE__,arg->uuid.val.uuid16);
    gattc_des_discover(arg->peer,arg->chr,&des0_uuid,ble_disc_2902_des_cbk,NULL);
    dtss_chr1 = arg->chr;
    peer_dev = arg->peer;
}
```

特征值描述符查找成功后，写特征值描述符，使能 Notify：

```
void ble_disc_2902_des_cbk(gap_event_t ev, void *ctx, gatt_chr_discovered_t *arg)
{
    co_printf("%s %d\r\n",__FUNCTION__,__LINE__);

    gattc_dsc_ccc_enable(arg->peer,dtss_chr1,GATT_SEND_VAL_NOTIFICATION,(gatt_cbk_t *)ble_disc_2902_des_en_cbk,NULL);
}
```

写特征值描述符成功后，DTSC 连接成功：

Baiot Confidential

```
void ble_disc_2902_des_en_cbk(gap_event_t ev, void *ctx, gatt_chr_val_notif_t *arg)
{
    co_printf("%s %d\r\n",__FUNCTION__,__LINE__);
    static uint8_t tem = 0;
    if(tem != 0){
        co_printf("%s %d\r\n",__FUNCTION__,__LINE__);
        app_usart_tx_fifo_enter(arg->val,arg->val_len);
    }
    tem = 1;
}
```

5.10.3. 蓝牙断开事件

```
void ble_gap_disconnected(gap_event_t ev, void *ctx, gap_disc_cmpl_t *arg)
{
    app_usart_dma_enable(DISABLE);

    co_printf("disconn complete, %08X, %02X\r\n", (uint32_t)arg->peer, arg->result);
}
```

5.11. DTSC 收发数据

蓝牙接收对端数据：

```
void ble_disc_2902_des_en_cbk(gap_event_t ev, void *ctx, gatt_chr_val_notif_t *arg)
{
    co_printf("%s %d\r\n",__FUNCTION__,__LINE__);
    static uint8_t tem = 0;
    if(tem != 0){
        co_printf("%s %d\r\n",__FUNCTION__,__LINE__);
        app_usart_tx_fifo_enter(arg->val,arg->val_len);
    }
    tem = 1;
}
```

发送蓝牙数据：

```
void usart_forward_to_ble_loop(void)
{
    uint32_t in_temp;
    uint16_t ble_send_len;

    in_temp = usart_rx_fifo_in;
    if(usart_rx_fifo_out < in_temp)
    {
        ble_send_len = in_temp-usart_rx_fifo_out;
    }
    else if(usart_rx_fifo_out > in_temp){
        ble_send_len = USART_RX_FIFO_SIZE-usart_rx_fifo_out;
    }
    else if(usart_rx_fifo_out == in_temp){
        // fifo empty, stop send loop
        ble_sending = false;
        return;
    }
    if(ble_send_len > ble_att_mtu)
    {
        ble_send_len = ble_att_mtu;
    }
    if(ble_send_len == 2)
    {
        if(memcmp( &usart_rx_fifo_buf[usart_rx_fifo_out],"\xfa\x55",2) == 0)
        {
            usart_rx_fifo_out = usart_rx_fifo_in = 0;
            return;
        }
    }

    gattc_chr_val_write_cmd(peer_dev,dtss_chr0,&usart_rx_fifo_buf[usart_rx_fifo_out], ble_send_len,NULL,NULL);
    usart_rx_fifo_out = (usart_rx_fifo_out+ble_send_len)%USART_RX_FIFO_SIZE;

    return;
}
```

Barrot Confidential

注：通过调用 gattc_chr_val_write_cmd()向 DTSS 端发送数据。

5.12. 任务与调度机

整个应用系统由多个独立的任务构成，运行于一个任务调度机之上，此调度机是一个具有优先级的，非实时的，不可阻塞的消息轮询处理机。

此调度机为 cs_schedule()函数，需要周期性的得到执行。运行于调度机内的任务原则上不可以一直占用处理器时间。

另一种编程方式是，由用户程序来分配处理器使用时间，并尽可能频繁的调用 cs_schedule()函数。

6.API 说明

6.1. ble 功能模块

更为详细的说明请参考《BARROT_BRLINK 数传类蓝牙 SDK 用户手册》。

6.1.1. le_adv_add

语法	void le_adv_add(const adv_param_t *par, gap_cbk_t *cbk, void *ctx);
参数	[in] par: 指向蓝牙广播参数的结构体指针 [in] cbk: 广播参数设置完成的回调函数 [in] ctx: 保留参数
返回	无
功能	创建蓝牙广播

Barrot Confidential

示例	adv_param_t adv_par = {0};
	
	adv_par.hdl = 0x01;
	adv_par.adv_ev_prop=
	(ADV_EV_PROP_BITS_LEGACY|ADV_EV_PROP_BITS_SCANNABLE |
	ADV_EV_PROP_BITS_CONNECTABLE);
	adv_par.prim_adv_intv_min = 0x20;
	adv_par.prim_adv_intv_max = 0x40;
	adv_par.prim_adv_ch_map = (ADV_CH_MAP_37 | ADV_CH_MAP_38 |
	ADV_CH_MAP_39);
	adv_par.own_addr_type = ADDR_TYPE_PUBLIC;
	adv_par.peer_addr_type = ADDR_TYPE_PUBLIC;
	adv_par.peer_addr = {0};
	adv_par.adv_filter_policy = ADV_POLICY_PROC_SCAN_CONN_ALL;
	adv_par.adv_tx_pwr = 0x7F;
	adv_par.prim_adv_phy = ADV_PHY_1M;
	adv_par.scnd_adv_max_skip = 0x00;
	adv_par.scnd_adv_phy = ADV)PHY_1M;
	adv_par.adv_sid = 0;
	adv_par.scan_req_notif_en = false;
	adv_par.adv_data_frag_pref = ADV_FRAG_PREF_CNTL_MAY_FRAG;
	adv_par.adv_data_frag_num = 0;

Parrot Confidential

	adv_par.adv_data_frag = NULL;
	adv_par.scn_data_frag_pref = ADV_FRAG_PREF_CNTL_MAY_FRAG;
	adv_par.scn_data_frag_num = 0;
	adv_par.scn_data_frag = NULL;
	le_adv_add(&adv_par,NULL,NULL);

6.1.2. le_adv_remove

语法	void le_adv_remove(uint8_t adv_hdl, gap_cbk_t *cbk, void *ctx);
参数	[in] adv_hdl: 需移除广播句柄 [in] cbk: 移除完成的回调函数 [in] ctx: 保留参数
返回	无
功能	移除指定的蓝牙广播
示例	le_adv_remove(0x01,NULL,NULL);

6.1.3. le_adv_clear

语法	void le_adv_clear(gap_cbk_t *cbk, void *ctx);
参数	[in] cbk: 清除所有广播完成的回调函数 [in] ctx: 保留参数
返回	无

Bariot Confidential

功能	清除所有广播
示例	le_adv_clear(NULL,NULL);

6.1.4. le_adv_enable

语法	void le_adv_enable(uint8_t adv_set_nm, const adv_en_param_t par, gap_cbk_t *cbk, void *ctx);
参数	[in] adv_set_num: 广播开启数量 [in] par: 广播开启参数，详见示例 [in] cbk: 广播开启完成后用户的回调函数 [in] ctx: 保留参数
返回	无
功能	开启指定数量的蓝牙广播

示例	adv_en_param_t en_par = {0};
	en_par.hdl = 0x01;
	en_par.duration = 0;
	en_par.events = 0;
	le_adv_en(1,&en_par,NULL,NULL)

6.1.5. le_adv_disable

语法	void le_adv_disable(uint8_t adv_set_num,_const uint8_t *adv_hdl, gap_cbk_t *cbk,

Barrot Confidential

	void *ctx);
参数	[in] adv_set_num: 广播关闭数量 [in] adv_hdl: 广播关闭句柄 [in] cbk: 广播关闭完成的回调函数 [in] ctx: 保留参数
返回	无
功能	关闭指定数量和句柄的蓝牙广播
示例	uint8_t adv_disen_hdl[] = {0x01} le_adv_en(1,adv_disen_hdl,NULL,NULL)

6.1.6. le_scan_enable

语法	void le_scan_enable(const scn_param_t *par, gap_cbk_t *cbk, void *ctx)
参数	[in] par: 见示例 [in] cbk: 扫描结果报告回调函数 [in] ctx: 保留参数
返回	--
功能	设置扫描参数并启动扫描
示例	scn_phy_param_t phy_par = { .scn_type = SCN_TYPE_ACTIVE,//主动扫描类型 .scn_intv = 160,//扫描间隔 160*0.625ms .scn_win= 150//扫描窗口 150*0.625ms

Bartot Confidential

```
    };
    scn_param_t san_par = {
        .filter_duplicates = false,//不过滤重复
        .duration = 0x0000,//持续扫描不停止
        .period   = 0x0000,//持续扫描不停止
        .filter_policy = SCN_POLICY_BASIC_UNFILTERED,
        .phys = SCH_PHY_BITS_1M,//扫描 1M PHY
        .phys_par = &phy_par,
    };
le_scan_enable(&san_par, (gap_cbk_t *)ble_scan_rsp_data_cbk, NULL);
```

6.1.7. le_scan_disable

语法	void le_scan_disable()
参数	无
返回	无
功能	停止扫描
示例	无

6.1.8. le_conn_create

语法	peer_t le_conn_create(const le_conn_par_t *par, gap_cbk_t *cbk, void *ctx)

Barrot Confidential

参数	[in] par:　创建连接的参数 [in] cbk:　连接完成的调函数 [in] ctx:　保留参数 typedef struct le_conn_par { 　　uint8_t　　init_policy; 　　uint8_t　　own_addr_type; 　　uint8_t　　peer_addr_type; 　　bd_addr_t　peer_addr; 　　uint8_t　　phys; 　　uint16_t　scn_intv; 　　uint16_t　scn_win; 　　uint16_t　conn_intv_min; 　　uint16_t　conn_intv_max; 　　uint16_t　latency_max; 　　uint16_t　sv_timeout; 　　uint16_t　ce_len_min; 　　uint16_t　ce_len_max; } le_conn_par_t;
返回	peer_t 指向连接资源的指针
功能	创建蓝牙连接
示例	无

Barrot Confidential

6.1.9. le_conn_destory

语法	void le_conn_destory(peer_t peer)
参数	[in] peer: 指向连接资源的指针
返回	无
功能	释放蓝牙连接
示例	无

6.1.10. gap_conn_create_by_addr

语法	peer_t gap_conn_create_by_addr(const bd_addr_t addr, addr_type_t type , gap_cbk_t *cbk, void *ctx)
参数	[in] addr: 目标设备蓝牙地址 [in] type: 目标设备蓝牙地址类型 [in] cbk: 回调函数 [in] ctx: 保留参数
返回	peer_t 指向此连接资源的指针
功能	通过地址创建蓝牙连接
示例	void ble_scan_rsp_data_cbk(gap_event_t ev, void *ctx, adv_rpt_t *arg) { uint8_t addr[] = {0x11,0x22,0x33,0x33,0x22,0x33}; if(memcmp(addr,&(arg->addr[0]),6) == 0)

Barrot Confidential

	{
	gap_conn_create_by_addr(arg->addr,arg->addr_type,
	ble_scan_dev_to_conn_cbk,NULL);
	}
	}

6.1.11. gap_ev_subscribe

语法	void gap_ev_subscribe(const gap_ev_cbk_t *ev_cbk, uint8_t nm)
参数	[in] ev_cbk: 事件与回调数组 [in] nm: 数组长度 回调函数类型 typedef void (gap_cbk_t)(gap_event_t ev, void *ctx, void *par); 事件与回调函数组合参数 typedef struct gap_ev_cbk { gap_event_t ev; gap_cbk_t *cbk; } gap_ev_cbk_t;
返回	无
功能	GAP 事件订阅

Barrot Confidential

示例	static gap_ev_cbk_t ble_gap_ev_cbk[] = {
	{GAP_EV_CONN_CMPL, (gap_cbk_t *)ble_gap_connected} ,
	{GAP_EV_DISC_CMPL, (gap_cbk_t *)ble_gap_disconnected} };
	
	int ble_gap_init(void)
	{
	gap_ev_subscribe(ble_gap_ev_cbk,
	sizeof(ble_gap_ev_cbk)/sizeof(gap_ev_cbk_t));
	return 0;
	}

6.1.12. gap_ev_unsubscribe

语法	void gap_ev_unsubscribe(const gap_ev_cbk_t *ev_cbk)
参数	[in] ev_cbk: 传入参数，取消对应事件与回调，参考示例
返回	无
功能	GAP 事件取消订阅
示例	无

6.1.13. gatts_svc_create

语法	gatt_svc_t gatts_svc_create(const uuid_t *uuid, gatt_cbk_t *ev_cbk, void *context)

Baiot Confidential

参数	[in] uuid: 指向 UUID 的指针 [in] ev_cbk: 服务回调函数 [in] context: 保留参数
返回	指向服务资源的指针
功能	创建 GATT 服务

示例	uuid_t svc_uuid = {
	UUID_LEN_16//16 位 UUID
	, UUID16_VAL_SET(0xFF00)//UUID,0xFF00
	};
	dtss_svc = gatts_svc_create(&svc_uuid, dtss_svc_cbk, NULL);

6.1.14. gatts_chr_add

语法	gatt_chr_t gatts_chr_add(gatt_svc_t svc, const uuid_t *uuid, uint16_t props , uint16_t val_perms, gatt_cbk_t *ev_cbk, void *context)
参数	[in] svc: 指向服务资源的指针 [in] uuid: 指向 UUID 的指针 [in] props: 特征值属性 [in] val_perms: 操作权限 [in] ev_cbk: 特征值状态变化回调函数 [in] context: 保留参数
返回	指向特征资源的指针

Baroit Confidential

功能	GATT 服务下添加特征值
示例	dtss_chr1 = gatts_chr_add(dtss_svc, &chr1_uuid , GATT_CHR_PROP_NOTIFY , ATT_PERM_READ , dtss_chr1_cbk, NULL);

6.1.15. gatts_dsc_ccc_add

语法	void gatts_dsc_ccc_add(gatt_chr_t chr, uint8_t props, uint16_t perms)
参数	[in] chr: 指向特征资源的指针 [in] props: 描述符属性 [in] perms: 操作权限
返回	无
功能	GATT 服务下添加特征值描述符
示例	gatts_dsc_ccc_add(dtss_chr1 , GATT_CHR_PROP_NOTIFY , ATT_PERM_READ | ATT_PERM_WRITE);

6.1.16. gatts_chr_val_write_rsp

语法	void gatts_chr_val_write_rsp(peer_t peer, gatt_chr_t chr, gatt_rst_t res);
参数	[in] peer: 指向目标连接资源指针

Baroit Confidential

	[in] chr: 回复的特征值 [in] res: 操作结果
返回	无
功能	特征值写入后响应

示例	void dtss_chr0_cbk(gatt_event_t event, void *context, void *param)
	{
	switch (event) {
	case GATTS_EV_CHR_VALUE_WRITE_REQ:
	gatt_chr_val_write_req_t *in = (gatt_chr_val_write_req_t *)param;
	gatts_chr_val_write_rsp(in->peer, in->chr, GATT_OK);
	}

6.1.17. gatts_chr_val_read_rsp

语法	void gatts_chr_val_read_rsp(peer_t peer, gatt_chr_t chr, gatt_rst_t res, uint8_t *val, uint16_t val_len)
参数	[in] peer: 指向目标连接资源指针 [in] chr: 指向特征的指针 [in] res: 操作结果 [in] val: 响应数据 [in] val_len: 响应数据长度
返回	无

Barrot Confidential

功能	服务端响应客户端读取特征操作
示例	无

6.1.18. gatts_chr_val_notif

语法	gatt_rst_t gatts_chr_val_notif(peer_t peer, gatt_chr_t chr, uint8_t *val, uint16_t val_len)
参数	[in] peer: 指向目标连接资源指针 [in] chr: 指向特征的指针 [in] val: 通知数据 [in] val_len: 通知数据长度
返回	无
功能	服务端发送通知数据到客户端
示例	无

6.1.19. gattc_svc_discover

语法	void gattc_svc_discover(peer_t peer, uuid_t *uuid, gatt_cbk_t *cbk, void *context)
参数	[in] peer: 指向目标连接资源指针 [in] uuid: 查找服务的 UUID [in] cbk: 获取服务完成的回调函数 [in] context: 保留参数

Barrot Confidential

返回	无
功能	获取服务端的服务。
示例	无

6.1.20. gattc_chr_discover

语法	void gattc_chr_discover(peer_t peer, gatt_svc_t *svc, uuid_t *uuid, gatt_cbk_t *cbk, void *context)
参数	[in] peer: 指向目标连接资源指针 [in] svc: 服务句柄 [in] uuid: 查找特征的 uuid [in] cbk: 获取服务端设备特征完成后的回调函数 [in] context: 保留参数
返回	无
功能	获取服务端设备的特征。
示例	无

6.1.21. gattc_des_discover

语法	void gattc_des_discover(peer_t peer, gatt_chr_t *chr, uuid_t *uuid , gatt_cbk_t *cbk, void *context)
参数	[in] peer: 指向目标连接资源指针

Bariot Confidential

	[in] chr: 指向特征的指针 [in] uuid: 查找 descriptor 的 uuid [in] cbk: 获取服务端设备 descriptor 完成后的回调函数 [in] context: 保留参数
返回	无
功能	获取服务端设备的描述符。
示例	无

6.1.22. gattc_dsc_ccc_disable

语法	void gattc_dsc_ccc_disable(peer_t peer, gatt_chr_t *chr)
参数	[in] peer: 指向目标连接资源指针 [in] chr: 指向特征的指针
返回	无
功能	失能服务端描述符。
示例	

6.1.23. gattc_dsc_ccc_enable

语法	void gattc_dsc_ccc_enable(peer_t peer, gatt_chr_t *chr, gatt_ccc_val_t val , gatt_cbk_t *cbk, void *context)
参数	[in] peer: 指向目标连接资源指针

Barrot Confidential

	[in] chr: 指向特征的指针 [in] val: 设置 descriptor 的值 GATT_SEND_VAL_NOTIFICATION 允许服务端设备通知 GATT_SEND_VAL_INDICATION 允许服务端设备指示 [in] cbk: 使能服务端描述符完成的回调函数及数据接收回调函数。 [in] context: 保留参数。
返回	无
功能	使能服务端描述符。
示例	无

6.1.24. gattc_chr_val_read

语法	void gattc_chr_val_read(peer_t peer, gatt_chr_t *chr, gatt_cbk_t *cbk, void *context)
参数	[in] peer: 指向目标连接资源指针 [in] chr: 指向特征的指针 [in] cbk: 读取特征值完成的回调函数 [in] context: 保留参数。
返回	无
功能	读取对端服务特征值
示例	无

Bariot Confidential

6.1.25. gattc_chr_val_write

语法	void gattc_chr_val_write(peer_t peer, gatt_chr_t *chr, uint8_t *val, uint16_t val_len, gatt_cbk_t *cbk, void *context)
参数	[in] peer: 指向目标连接资源指针 [in] chr: 指向特征的指针 [in] val: 指向写入数据的指针 [in] val_len: 指向写入数据的长度 [in] cbk: 数据写入完成的回调函数 [in] context: 保留参数
返回	无
功能	向服务端特征值发送数据待对端回复
示例	无

6.1.26. gattc_chr_val_write_cmd

语法	void gattc_chr_val_write_cmd(peer_t peer, gatt_chr_t *chr, uint8_t *val, uint16_t val_len, gatt_cbk_t *cbk, void *context)
参数	[in] peer: 指向目标连接资源指针 [in] chr: 指向特征的指针 [in] val: 指向写入数据的指针 [in] val_len: 指向写入数据的长度

Bartot Confidential

	[in] cbk: 数据写入完成的回调函数 [in] context: 保留参数
返回	无
功能	向服务端特征值发送数据且无需响应
示例	无

6.2.2.4G 功能模块

6.2.1. direct_24g_init

语法	int direct_24g_init(cbk_direct_24g_tx tx_cbk ,cbk_direct_24g_rx rx_cbk ,cbk_direct_24g_rx_timeout rx_timeout_cbk ,uint8_t access_code[4] ,direct_24g_mode mode);
参数	[in] cbk_direct_24g_tx tx_cbk: TX 回调函数见 7.2.4 [in] cbk_direct_24g_rx rx_cbk: RX 回调函数见 7.2.5 [in] cbk_direct_24g_rx_timeout rx_timeout_cbk: RX timeout 回调函数见 7.2.6 [in] uint8_t access_code[4]: [in] direct_24g_mode mode:TX/RX 模式
返回	错误代码，见 7.2.7
功能	2.4G 私有协议初始化
示例	无

Barrot Confidential

6.2.2.direct_24g_tx

语法	int direct_24g_tx(uint8_t access_code[4], uint16_t freq, const uint8_t *data, uint8_t len)
参数	[in] uint8_t access_code[4]: [in] uint16_t freq: 工作信道 [in] const uint8_t *data: 待发送数据位置 [in] uint8_t data_len: 待发送数据长度，单位字节
返回	错误代码
功能	2.4G 私有协议发送数据函数
示例	无

6.2.3.direct_24g_rx

语法	int direct_24g_rx(uint8_t access_code[4],uint16_t freq, uint16_t rx_win_size)
参数	[in] uint8_t access_code[4]: [in] uint16_t freq: 工作信道 [in] uint16_t rx_win_sz: 接收时间窗口，单位毫秒
返回	错误代码
功能	2.4G 私有协议接收数据函数
示例	无

Barrot Confidential

6.2.4.cbk_direct_24g_tx

语法	typedef void (*cbk_direct_24g_tx)(uint16_t freq);
参数	[in] uint16_t freq: 工作信道
返回	无
功能	2.4G 私有协议 TX 完成事件回调函数
示例	无

6.2.5. cbk_direct_24g_rx

语法	typedef void (*cbk_direct_24g_rx)(uint16_t freq,uint8_t rssi,void *param, uint8_t param_len);
参数	[in] uint16_t freq: 工作信道 [in] uint8_t rssi: 接收信号强度指示 [in] void *param: 参数指针，接收时转换为 uint8_t * [in] uint8_t param_len: 接收数据长度，单位字节
返回	无
功能	2.4G 私有协议 RX 完成事件回调函数
示例	无

6.2.6. cbk_direct_24g_rx_timeout

Barrot Confidential

语法	typedef void (*cbk_direct_24g_rx_timeout)(uint16_t freq);
参数	[in] uint16_t freq: 工作信道
返回	无
功能	2.4G 私有协议 RX 超时事件回调函数
示例	无

6.2.7.错误代码

宏定义	数值	说明
D24G_ERRCODE_SUCCESS	0	无错误
D24G_ERRCODE_NOINIT	-1	未初始化
D24G_ERRCODE_INVALID_PARAM	-2	非法参数
D24G_ERRCODE_STATE_ERR	-3	状态异常
D24G_ERRCODE_SCH_ERR	-4	调度异常
D24G_ERRCODE_RESOURCE_LIMITED	-5	资源受限

6.3.调试信息模块

6.3.1. co_log_init

语法	void co_log_init(void)
参数	无
返回	无

Barrot Confidential

功能	初始化调试信息打印模块，默认使用:PB1，波特率 115200 用户可根据实际需要选择其他串口输出。
示例	无

6.3.2.co_printf

语法	int co_printf(const char *fmt, ...)
参数	[in] fmt：输出格式 [in] ...：可变参数
返回	成功打印出字符串长度，不含结束符'\0'
功能	格式化打印输出函数，语法使用和 printf 函数相同。
示例	无

6.3.3. co_printf_type

语法	int co_printf_type(uint16_t dbg_type, const char *fmt, ...)
参数	[in] dbg_type：输出类型 [in] fmt：输出格式 [in] ...：可变参数
返回	成功打印出字符串长度，不含结束符'\0'
功能	带类型的格式化打印输出函数，语法使用和 printf 函数相同。
示例	无

Barrot Confidential

6.3.4. dbg_type

宏定义	数值	说明
DBG_TYPE_NONE	0x0000	关闭全部信息
DBG_TYPE_ERROR	0x0001	开启错误信息输出
DBG_TYPE_WARNING	0x0002	开启警告信息输出
DBG_TYPE_INFO	0x0004	开启一般信息输出
DBG_TYPE_DRIVER	0x0008	开启驱动信息输出
DBG_TYPE_CS	0x0010	开启通讯系统信息输出
DBG_TYPE_APP	0x0100	开启 APP 信息输出
DBG_TYPE_ALL	0xFFFF	开启全部信息输出

6.4.休眠管理模块

6.4.1. br_sleep_lock_acquire

语法	uint8_t br_sleep_lock_acquire(void)
参数	无
返回	true：上锁成功 false：上锁溢出
功能	睡眠功能上锁，防止系统进入休眠，这是一个计数锁
示例	无

Barrot Confidential

6.4.2.br_sleep_lock_release

语法	uint8_t br_sleep_lock_release(void)
参数	无
返回	true: 释放成功 false: 已全部释放
功能	睡眠功能解锁
示例	无

6.4.3. br_sleep

语法	void br_sleep(void)
参数	无
返回	无
功能	休眠管理函数

示例	while(1)
	{
	cs_schedule();
	other_task();
	br_sleep();
	}

Barrot Confidential

6.4.4. app_sleep_prepare_proc

语法	void app_sleep_prepare_proc(void)
参数	无
返回	无
功能	用户定义的进入休眠前的处理函数，只能使用这个名字
示例	无

6.4.5. app_sleep_resume_proc

语法	void app_sleep_prepare_proc(void)
参数	无
返回	无
功能	用户定义的退出休眠后立即处理函数，只能使用这个名字
示例	无

6.5.阻塞延迟模块

简介：阻塞延迟函数模块，里面声明的相关函数具体实现在 ROM 代码，这里只是声明，用户代码添加头文件之后可以直接调用。应注意其延迟的时间并不精准，如需精准延迟建议使用硬件定时器功能。

源码文件：br_delay.h

Barrot Confidential

6.5.1. delay_cycles

语法	void delay_cycles(uint32_t ui32Cycles);
参数	[in] ui32Cycles：需要等待的 cycles 数量，一个 cycles 的时间约为(10/110) 微秒。
返回	无
功能	延迟等待指定数量的 cycles 返回。
示例	delay_cycles(1000);

6.5.2. delay_n_us

语法	void delay_n_us(uint32_t val);
参数	[in] val：需要等待的微秒数。
返回	无
功能	延迟等待指定数量的微秒。
示例	delay_n_us(1000);

6.5.3. delay_n_10us

语法	void delay_n_10us(uint32_t val);
参数	[in] val：需要等待的 10 微秒数。
返回	无
功能	延迟等待指定数量的 10 微秒。

Barrot Confidential

示例	delay_n_10us(1000);

6.5.4. delay_n_100us

语法	void delay_n_100us(uint32_t val);
参数	[in] val：需要等待的 100 微秒数。
返回	无
功能	延迟等待指定数量的 100 微秒。
示例	delay_n_100us(1000);

6.5.5. delay_n_ms

语法	void delay_n_ms(uint32_t val);
参数	[in] val：需要等待的毫秒数。
返回	无
功能	延迟等待指定数量的毫秒。
示例	delay_n_ms(1000);

6.6.杂类接口

6.6.1. 查询 SDK 版本

语法	void cs_get_sdk_version(void);

Barrot Confidential

参数	无。
返回	版本字符串。
功能	返回 SDK 库的版本。
示例	举例：“BR1601x_LIB_0000_250218_r4047”

7. 公司简介

百瑞互联——世界领先的拥有自主知识产权的一站式芯片级无线互联物联网方案提供商，是蓝牙国际组织（Bluetooth SIG）的高级会员，国内唯一的蓝牙标准制定公司，拥有自主知识产权的蓝牙射频、协议栈、音频算法三大 IP，聚焦物联网和汽车电子两大领域。

致力成为全球最值得信赖的无线短距离通信公司！

8.联系我们

8.1. 北京

联系电话: +86 10 82702580

Fax: +86 10 82898219

地址: 北京市海淀区上地三街 9 号嘉华大厦 A 座 A1009

市场邮件: marketing@barrot.com.cn

技术支持: support@barrot.com.cn

网址: www.barrot.com.cn

8.2.深圳

联系电话: +86 755-33170692

Barrot Confidential

地址:广东省深圳市宝安区新安街道兴东社区 67 区中粮商务公园 1 栋 5 层

技术支持: support@barrot.com.cn

网址: www.barrot.com.cn

8.3.上海

地址: 上海市浦东新区张江高科碧波路 500 号 2 层

技术支持: support@barrot.com.cn

网址: www.barrot.com.cn

9.版权

版权 ©2023 北京百瑞互联技术股份有限公司

版权所有。

蓝牙商标由美国 Bluetooth SIG Inc.拥有，并许可北京百瑞互联技术股份有限公司使用。

本文中列出的所有其他商标均由其各自所有者拥有。

Barrot Confidential